

ESOF 322

Software Engineering

LECTURE: 9

J. L. Pach

OUTLINE PR

1. What is a Pull Request?
2. Why Do We Use PRs?
3. The Author's Workflow
4. Writing a *Good* PR Description
5. Example: Creating a PR
6. **The Reviewer's Workflow**
7. **The Verdict: Approve, Comment, or Request Changes**
8. In-Class Exercise

WHAT IS A PULL REQUEST?

> A Pull Request (PR) is a formal **request to pull your changes** from your branch into another branch (usually `main`).

It is not just a command. It is the **center of discussion about code**.

- It is a **formal proposal** to the team: "My work is done, here is the code, please check it."
- It is the **quality gate** protecting the `main` branch from bugs and low-quality code.
- It is a **historical record** of why a change was made.

Why Do We Use PRs?

1. QUALITY ASSURANCE

It is the last line of defense. Before code is merged, it is reviewed by at least one other person. This is where we catch bugs, logic errors, and style issues.

2. KNOWLEDGE TRANSFER

- The **reviewer** learns how a new part of the system works.
- The **author** learns how others perceive their code and what could be improved.
- The whole team stays aware of what is happening in the project.

3. TRACEABILITY

Years from now, when you wonder "Why was this function written in such a strange way?", you can find the **PR** and read the entire discussion and justification.

THE AUTHOR'S WORKFLOW

1. **Create a branch** for your task (`git checkout -b ...`).
2. **Write code** and make small, logical commits.
3. **Push your branch** to GitHub (`git push ...`).
4. **Open a Pull Request** on GitHub's website.
5. **Write a clear Title and Description** for your PR.
6. **Assign a Reviewer** from the panel on the right. This is mandatory. The reviewer will get an email notification.

THE ANATOMY OF A PULL REQUEST

A PR on GitHub has a few key components:

- **Title:** A short, clear summary of the change.
- **Description:** A detailed explanation of what you did, why you did it, and how to test it.
- **Reviewers:** The people you are asking to check your code.
- `Conversation`: The main discussion area.
- `Commits`: A list of all commits in the PR.
- `Files Changed`: The most important part for a reviewer. It shows the "diff" — all the lines of code that were added, removed, or modified.

THE FULL WORKFLOW

Here is the standard sequence of events to create a PR.

```
# 1. Make sure your local main is up-to-date
git checkout main
git pull origin main

# 2. Create a new branch for your task
git checkout -b feat/calculate-average

# 3. Write your code, make small, logical commits
# ... write code ...
git add utils.c
git commit -m "Feat: Add function to calculate average"

# 4. Push your branch to the remote repository
git push -u origin feat/calculate-average

# 5. On GitHub: Go to the repository page. You will see a yellow banner with a "Compare & pull request" button. Click it.
```

Writing a GOOD PR Description

A blank description is not acceptable. It wastes the reviewer's time. Use this template:

> **WHAT WAS DONE?**
>
> - *A bulleted list of your changes.*
> - *Explain what you accomplished.*
>
> **HOW TO TEST?**
>
> 1. *Step-by-step instructions for the reviewer.*
> 2. *Tell them what to do and what to look for.*
> 3. *Example: "Call* `CalcAverage(10, 20)` *. The expected result is 15."*
>
> **KNOWN LIMITATIONS**
>
> - *Is there anything this change does NOT do?*
> - *Example: "The function does not yet handle negative numbers."*

EXAMPLE

A Simple Averaging Function

THE TASK

You are asked to write a C function that calculates the average of three integers.

You create a new branch `feat/calculate-average` and add the following function to a `utils.c` file.

THE "BAD" INITIAL CODE

You write this function. It seems to work, but it has a hidden, critical bug.

```c
#include <stdio.h>

/*
 * Calculates the average of three integers and prints it.
 * Returns 0 on success.
 */
int calculateAverage(int a, int b, int c) {
    int avg = (a + b + c) / 3;
    printf("The average is: %d\n", avg);
    return 0;
}
```

The bug is integer division. For inputs `(8, 8, 9)`, the sum is 25. `25 / 3` in integer math is `8`, not `8.33`. This is a loss of precision.

The "Good" Refactored Code

You improve the code to fix the bug and make the function more useful.

```
#include <stdio.h>

/*
- * Calculates the average of three integers and prints it.
- * Returns 0 on success.
+ * Calculates the average of three integers.
+ * Returns the average as a float.
 */
- int calculateAverage(int a, int b, int c) {
-     int avg = (a + b + c) / 3;
-     printf("The average is: %d\n", avg);
-     return 0;
+ float calculateAverage(int a, int b, int c) {
+     float avg = (a + b + c) / 3.0f;
+     return avg;
}
```

```
// The responsibility of printing is moved to main()
// int main() {
//     float result = calculateAverage(8, 8, 9);
//     printf("The average is: %f\n", result);
//     return 0;
// }
```

Now the function correctly calculates the average and returns it, which is much more flexible. You are ready to create a PR.

Creating the PR

You push your `feat/calculate-average` branch and click "Compare & pull request". You fill out the form carefully.

Title: `Fix: Correct integer division in average calculation`

Description:

WHAT WAS DONE?

- Changed the `calculateAverage` function to return a `float` instead of an `int` to prevent loss of precision from integer division.
- Removed the `printf` call from the function. The function should only calculate, not print. This improves reusability.
- Updated function comments to reflect the new return type.

HOW TO TEST?

1. Call `calculateAverage(8, 8, 9)`.
2. The expected result is `8.333...`, not `8`.

KNOWN LIMITATIONS

- The function currently only accepts 3 arguments. A future improvement could be to accept an array of numbers.

OUTLINE CR

1. What code review is and why it exists
2. Where CR fits in the Scrum cycle
3. The flow: from branch to `main`
4. Writing a good Pull Request
5. Writing and receiving review comments
6. Reviewer checklist
7. Common mistakes
8. Sequence in our course + exercise

WHAT IS CODE REVIEW?

> A systematic examination of source code by **at least one person other than the author**, before the change is merged into the main branch.

What it is NOT:

- an evaluation of the programmer as a person;
- finding mistakes to embarrass someone;
- a formality to check off before merging;

What it IS:

- a conversation about code quality;
- knowledge transfer within the team;
- the last safety net before `main`;

WHY WE DO IT — THREE REASONS

1. Quality

Defects found in review cost 10× less than defects found after deployment.

(Boehm 1981 — defect cost curve; still valid today)

2. Knowledge Transfer

The reviewer learns how the module works.

The author learns how others read their code.

3. Team Consistency

One quality standard across the entire project.

No one is the sole "owner" of a module — everyone understands it.

CODE REVIEW IN THE SCRUM CYCLE

```
Sprint Planning
      ↓
  Development        ← you write code in a feature branch
      ↓
  Code Review        ← Pull Request + review
      ↓
Definition of Done   ← only after review
      ↓
  Sprint Review
      ↓
Retrospective
```

> No change reaches `main` without a review.
>
> This is a **rule**, not a suggestion.

DEFINITION OF DONE — WHAT DOES "DONE" MEAN?

A task is **done** when **all** of these are true:

- [ ] Code compiles without errors or warnings
- [ ] All acceptance criteria are met
- [ ] Branch is pushed to remote
- [ ] Pull Request is open with a description
- [ ] At least one reviewer has approved
- [ ] All comments resolved or rejected with explanation
- [ ] Merged to `main`

> "Works on my machine" ≠ Done

PULL REQUEST FLOW — THE FULL SEQUENCE

1. git checkout -b feature/name-task
2. [write code, commit in small steps]
3. git push origin feature/name-task
4. GitHub → "Compare & pull request"
5. Fill in title and description
6. Assign a reviewer
7. Reviewer: Files changed → inline comments on code lines
8. Author: replies, pushes fixes to the same branch
9. Reviewer: Approve
10. Squash and merge → branch deleted

Every push to the same branch **updates the PR automatically**.

The Reviewer's Workflow

Once you are assigned, go to the `Files changed` tab. When you comment on a line:

Option 1: "Add single comment"

- Publishes your comment **immediately**.
- The author gets a notification right away.

> **Warning:** This can "spam" the author with many notifications if you have a lot of comments.

Option 2: "Start a review"

- Your comments are marked as **"Pending"**. Only you can see them.
- You can add many pending comments across multiple files.
- When you are finished, you submit all comments at once.
- **This is the recommended, professional method.**

The Verdict: The Final Step of a Review

After adding all your pending comments, click the **"Submit review"** button. You must choose one of three verdicts:

✅ Approve

- ⚒ "The code is good. I give my green light to merge."
- ⚒ Use when there are no issues, or only optional "nits".

💬 Comment

- ⚒ "I have some feedback, but I am not blocking the merge."
- ⚒ Use for general questions or observations that don't require changes.

❌ Request changes

- ⚒ "This code has problems that **must** be fixed before merging."
- ⚒ Use when you find a bug or a serious quality issue (🔴 `must`).

WRITING A GOOD PR — TEMPLATE

```
Title: Fix integer division in grade average function

## What was done
- Changed int avg to float avg (fix for rounding error)
- Added input range validation (0–100)
- Extracted print logic to PrintLetterGrade()
- Replaced magic numbers with #define constants

## How to test
1. CalcAverage(89, 90, 90) — should return B, not C
2. CalcAverage(-1, 50, 80) — should handle invalid input
3. CalcAverage(100, 100, 100) — should return A

## Known limitations
- Does not handle weighted grades yet (next task)
```

RULES FOR WRITING COMMENTS

Rule 1 — Comment on the code, not the person

✗ "this is unreadable"

✓ "doGrade does two things — consider extracting PrintLetterGrade()"

Rule 2 — Justify every comment

✗ "change this to a constant"

✓ "use #define GRADE_A 90 — when thresholds change, one place to update"

Rule 3 — Label the severity

Label	Meaning	Example
🔴 must	logic error, crash, unmet AC	rounding error changes the grade
🟡 should	quality, readability, maintainability	magic number instead of constant
🔵 nit	style, preference, optional	declaration order in file

ETYMOLOGY: WHY NIT?

- Nits are tiny, numerous, and tedious to remove one by one from someone's hair
- **Nitpicking** (US slang, attested ~1950s) — literally: combing through hair searching for nits
  - meticulous
  - low stakes
  - easy to lose perspective on what actually matters
- Metaphorical meaning (mid-20th century onward):

> *obsessing over trivial details while the big picture is fine*

FROM HAIR TO CODE REVIEW

Software engineering culture (Google engineering practices, GitHub-style workflows) borrowed the word as a **label**, not an insult:

- 🔵 **nit** = "I noticed this, but it's cosmetic"
- It protects **both sides** of the review:
  - the reviewer doesn't have to justify a minor taste preference
  - the author doesn't have to defend or fix it to get approval
- Contrast with 🔴 **must** — a real louse *bites*; a real bug *breaks things*.

  A nit is just... there. Annoying, optional to remove.

CODE TO REVIEW — THE EXAMPLE

A teammate pushes the following function:

```c
int doGrade(int a, int b, int c)
{
    int avg = (a + b + c) / 3;
    if (avg >= 90) printf("A");
    else if (avg >= 80) printf("B");
    else if (avg >= 70) printf("C");
    else           printf("F");
    return avg;
}
```



The next three slides show three comments on this code.

Before moving on: **what do you see?**

COMMENT 1 — LOGIC ERROR 🔴

```
int avg = (a + b + c) / 3;
```

❌ **Bad:** "wrong division"

✓ **Good:**

🔴 must: integer division truncates the fractional part.

Example: doGrade(89, 90, 90) → 269 / 3 = 89 (int) → grade C.

Correct grade: B (average 89.67).

Fix:

```
float avg = (a + b + c) / 3.0f;
```

COMMENT 2 — MAGIC NUMBERS

```c
if (avg >= 90) printf("A");
else if (avg >= 80) printf("B");
else if (avg >= 70) printf("C");
```



❌ **Bad:** "use constants"

✓ **Good:**

should: grade thresholds are magic numbers.

When thresholds change — three places to update, risk of missing one.

Suggest:

```c
#define GRADE_A 90
#define GRADE_B 80
#define GRADE_C 70
```



COMMENT 3 — POSITIVE + NIT 🔵

```c
int doGrade(int a, int b, int c) {
    ...
    return avg;
}
```



✓ Positive comment:

🟡 should: good that the function returns avg —

it can be reused without recalculating. Sound design decision.

✓ Nit:

🔵 nit: parameter names a, b, c do not describe what they represent.

Consider: score1, score2, score3.

In three months you will save 30 seconds of confusion.

RESPONDING TO COMMENTS

Situation	What you do
You agree	Fix the code, reply: *"Fixed in commit abc1234"*
You disagree	Explain **why** — leave it as is
You don't understand	Ask for clarification — this is normal
You partially agree	Describe what you changed and what you kept, with reason

> **Never mark a comment "Resolved"**
> if you ignored it without explanation.

REVIEWER CHECKLIST

must — correctness:

- Code compiles without warnings
- All acceptance criteria are met
- Edge cases handled: 0, negative values, maximum
- No: division by zero, overflow, off-by-one

should — quality:

- File header with authorship declaration *(required by syllabus!)*
- No magic numbers — use `#define`
- Each function does one thing
- Variable names describe purpose, not type

nit — style:

- Comments explain "why", not "what"
- Consistent indentation and formatting

COMMON MISTAKES — AUTHOR

🔴 **No PR description**

The reviewer does not know what to check — review becomes superficial

🟡 **Defensive attitude toward comments**

Code review is not an attack on you.

Your code is not you.

🔵 **Ignoring comments without a reply**

Every comment requires a response — even if you disagree.

Silence = ignoring = bad habit.

COMMON MISTAKES — REVIEWER

🔴 **30-second review**

`"looks good"` without reading the code — this is **not** a review

🔴 **Vague comments**

`"unreadable"` without specifying what and why — author does not know what to change

🟡 **Only negatives, no positives**

One comment like "elegant solution" builds team culture

🔵 **Style dictator**

`"do it my way"` vs `"this has problem X — one way to fix it is Y"`

SEQUENCE IN OUR COURSE

STEP 1 — Individual
  Each student writes in their own branch
  Push — even non-working code
        ↓

STEP 2 — Within-team review
  You review a teammate's branch
  3 comments → discussion → shared team version
        ↓

STEP 3 — Cross-team review
  Your team reviews the other team — and vice versa
        ↓

STEP 4 — In-class synthesis → main

THANK

You